MateMate architecture proposal

Unearth Your Solutions / Queen&King Inc. / SE4 subsystem establishment

MateMate is a chess application for one human playing against the computer. The player moves pieces by drag and drop. MateMate explores legal replies, evaluates future positions and displays its selected reply on the same board.

Scope and design assumptions. Use one offline desktop process, one active match in memory and a bounded background search. The opening walkthrough assumes the human plays White. The rules and engine handle both colors. Saving games, online play, tournament clocks and draw-claim UI are outside this proposal.

Subsystem structure

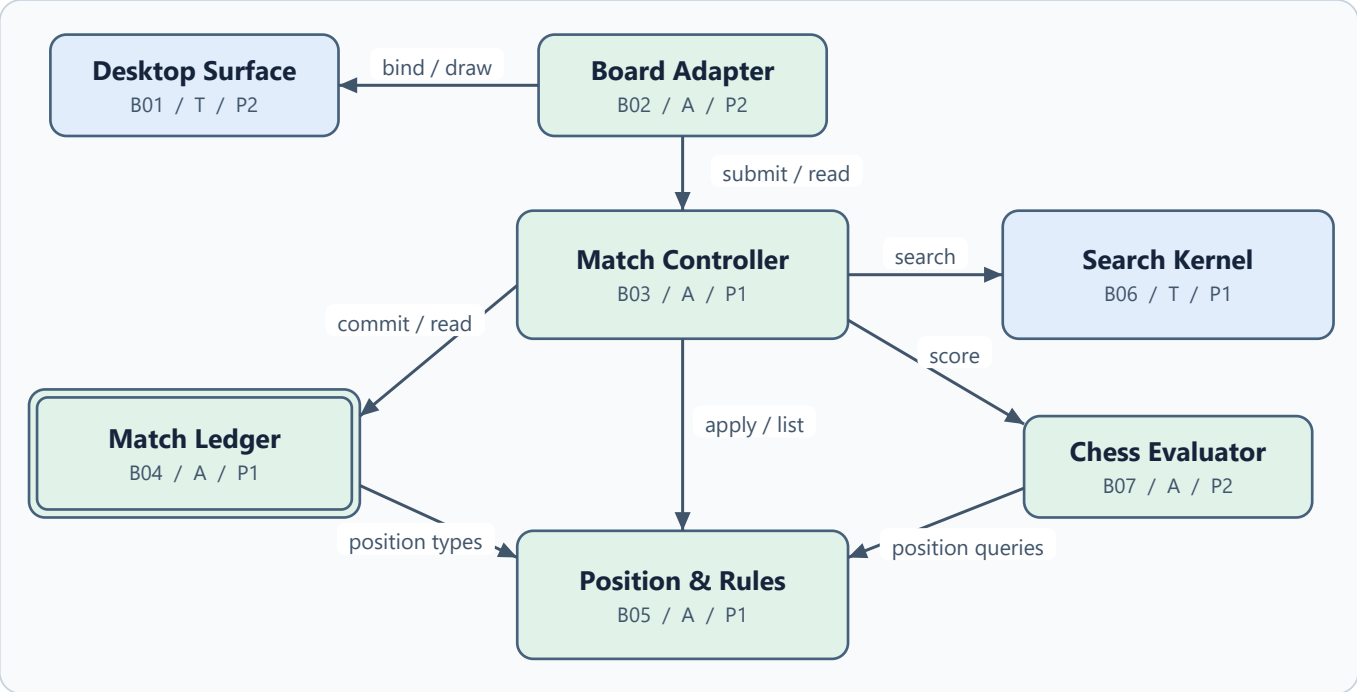


Diagram key. Green/A = chess-specific. Blue/T = reusable technical code. A double border marks committed game-state ownership. Large/P1 marks rules or turn-critical blocks and standard/P2 marks supporting blocks. Arrows point from the caller to the allowed provider.

Why these boundaries?

The search algorithm can be reused for another two-player game through generic callbacks. Chess puzzles can reuse Position & Rules, Chess Evaluator and Search Kernel independently of the match ledger. The board adapter is the only component that translates pixels into chess squares.

At runtime the GUI thread renders immutable snapshots. A serialized controller handles game commands. A worker searches snapshots without holding the ledger or changing the live board.

Subsystems and services

Responsibilities, owned knowledge and the permitted dependencies between subsystems.

Subsystem	Responsibility	Knowledge	May use
B01 Desktop Surface T	Pointer events, pixel primitives and the GUI event loop.	Widget geometry, latest raw click/drop and drawing primitives.	-
B02 Board Adapter A	Translates board gestures into square moves and renders chess snapshots.	Board orientation, active drag and piece layout.	B01, B03
B03 Match Controller A	Coordinates human and bot turns, legality checks and view updates.	Turn phase, active search token and serialized command handling.	B04, B05, B06, B07
B04 Match Ledger A	Owns the committed position, accepted moves and repetition history.	Match ID, revision, move records and historical snapshots.	B05
B05 Position & Rules A	Models a chess position and defines legal transitions and outcomes.	Board, side to move, castling, en passant and move counters.	-
B06 Search Kernel T	Selects a candidate through bounded minimax with alpha-beta pruning.	Generic search state, deadline, node budget and explored tree.	-
B07 Chess Evaluator A	Scores nonterminal chess positions from White's perspective.	Piece weights and positional features.	B05

Service allocation from the class brief

Provider	Elicited services and their ownership
B01 Desktop Surface	1 latest click in pixels, 5 draw a dot, 16 latest drop in pixels.
B02 Board Adapter	3 draw a pawn on a square, 6 draw a position, 12 piece display size, 13 latest click as a square, 18 draw an empty board.
B03 Match Controller	2 optimal-move service. B03 wires chess-specific callbacks and delegates the bounded search to B06.
B04 Match Ledger	15 whether Black has castled, 17 historical board, 19 Black's most recent move, 20 match age in accepted plies.
B05 Position & Rules	4 legal rook destinations, 8 White checkmated, 9 side to move, 11 White castling eligibility, 14 occupant of b2.
B07 Chess Evaluator	7 value for White, 10 rook weight. B06 uses scores to rank legal branches.

B05 supplies position facts and rule checks. For the current match, B03 reads the position from B04 and exposes view data to B02. Unlisted inter-subsystem calls are forbidden. Callback invocation by B01 or B06 uses their own interfaces and adds no concrete dependency.

Walkthrough 1 · Make move

A new game starts with White to move. The human drops the e2 pawn at screen position (824, 535).

Resolve the drag

For the walkthrough, assume a White-oriented 640 × 640 board at screen origin (500, 175), with 80-pixel squares and the y-axis increasing downward. The press already selected e2. At drop, the column is floor((824 - 500) / 80) = 4 and the row is floor((535 - 175) / 80) = 4, both zero-based. Column 4 is file e and row 4 is rank 8 - 4 = 4. The drop therefore requests **e2-e4**.

Step	Call and data	Result
1	B01 gives B02 the press and drop events. B02 checks the board bounds and uses the orientation and layout captured for the drag.	B02 submits **MoveIntent(e2, e4)** with the match ID and revision. A changed layout cancels the drag.
2	B03 checks the match ID, revision and human-turn phase, then asks B05 to validate and apply the intent to B04's snapshot.	B05 returns a new immutable position. B04 appends the move once. B02 renders White's move through B01.
3	B03 schedules B06 with the new position, a deadline, a legal fallback move and callbacks for legal transitions, terminal results and evaluation.	B06 uses bounded minimax. White maximizes, Black minimizes. B07 scores the nonterminal leaves and B05 supplies legal moves and terminal outcomes.
4	B06 returns a candidate for the same match ID, revision and search token. B03 checks all three and validates the move with B05 before asking B04 to append it.	B02 draws the committed reply from the returned board snapshot. For example, if the selected move is e7-e5, e7 becomes empty and the pawn appears at e5.

Interface and failure behavior

MoveIntent carries source square, target square and an optional promotion choice. **Snapshot** carries match ID, revision, phase, pieces, side to move and outcome. The UI refreshes on the GUI thread using the newest snapshot. Pixel coordinates stay in B01 and B02.

An illegal move leaves position and revision intact and snaps the piece back. A search timeout returns the best move from the last completed depth, or the precomputed legal fallback. A late or duplicate result is discarded. A terminal position skips reply search. If validation rejects the engine candidate, the controller tries the legal fallback once and reports a recoverable error if that also fails.

Walkthrough 2 · Castle

The same Position & Rules service checks a human move, an engine candidate and every hypothetical search branch.

Side	King move	Rook move	Required flag
White kingside	e1 to g1	h1 to f1	WK
White queenside	e1 to c1	a1 to d1	WQ
Black kingside	e8 to g8	h8 to f8	BK
Black queenside	e8 to c8	a8 to d8	BQ

A normal starting position has all four rights. A king move clears both rights for its color. An original rook moving from its home square clears that rook's right. Capturing an original rook on its home square clears that side's corresponding right. Rights can only be cleared during play. Returning a king or rook to its home square, or putting a promoted rook there, can never restore a flag.

B05 also verifies the side to move, the king and eligible rook on their home squares, an empty path, and that the king's start, transit and destination squares are safe from attack. A successful castle moves both pieces in one immutable transition. B04 appends one move and records that the side has castled. A rejected castle leaves the position and ledger revision unchanged.

Review fixtures. Try each of the four legal castles. For both colors, try again after the king or relevant rook leaves and returns. Try a captured original rook replaced by another rook, a blocked path and an attacked transit square. Confirm the rejected moves preserve the ledger revision.

State and search decisions

Immutable positions, separate history. A position contains only the board and rule-relevant state. B04 owns the accepted move list, historical snapshots and repetition counts. Search carries a private branch count seeded by B04. Repetition identity includes board, side to move, castling rights and a legally available en passant capture. Move counters are separate.

Single writer, bounded worker. B03 serializes commands and B04 is the sole writer of committed match state. The background worker searches immutable snapshots. This avoids undo bookkeeping and stale writes, at the cost of snapshot allocations and bounded search depth.

Sources

Martin Hasitschka, *Architecture Documentation*, V704, PDF pp. 5, 9-10 and 72.

Martin Hasitschka, *Architecture Development*, V702, PDF pp. 27-36 (MateMate and SE4).

FIDE Laws of Chess, arts. 3.8-3.9 and 9.2-9.6.